

Java从入门到精通

职业技能培训知识库

职业名称：Java开发工程师

版本：V1.0

第1章 Java基础入门

1.1 Java开发环境搭建

知识点讲解

Java是一种面向对象的编程语言，具有跨平台、安全性高、稳定性强等特点。在进行Java开发之前，必须正确搭建开发环境，包括JDK（Java Development Kit）的安装和环境变量的配置。JDK是Java开发工具包，包含了Java编译器（javac）、Java运行时环境（JRE）以及常用的开发工具。目前企业主流使用的是JDK 8和JDK 11，JDK 17也逐渐被采用。

操作步骤

Windows系统安装步骤：

1. 访问Oracle官网或OpenJDK官网，下载对应版本的JDK安装包
2. 运行安装程序，建议安装路径不要包含中文和空格，如 C:\Program Files\Java\jdk-1.8
3. 安装完成后，右键"此电脑"→"属性"→"高级系统设置"→"环境变量"
4. 新建系统变量 JAVA_HOME，值为JDK安装路径
5. 编辑系统变量 Path，添加 %JAVA_HOME%\bin
6. 打开CMD命令行，输入 java -version 验证安装是否成功

Linux系统安装步骤：

1. 使用包管理器安装：sudo apt-get install openjdk-8-jdk（Ubuntu）或 yum install java-1.8.0-openjdk（CentOS）
2. 编辑 /etc/profile 文件，添加：export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64; export PATH=\$PATH:\$JAVA_HOME/bin
3. 执行 source /etc/profile 使配置生效
4. 输入 java -version 验证

注意事项

注意：

- JDK和JRE的区别：JDK包含JRE，JRE仅包含运行时环境，不能用于编译代码
- 安装路径避免使用中文、空格和特殊字符，否则可能导致命令行工具无法识别
- 多个JDK版本共存时，通过修改JAVA_HOME来切换版本
- 环境变量配置后需要重新打开命令行窗口才能生效

常见问题

Q1：输入java -version提示"不是内部或外部命令"

A：说明环境变量未配置正确，检查Path变量中是否包含JDK的bin目录路径。

Q2: 安装后无法编译Java文件

A: 检查是否安装了JDK而非仅安装了JRE, javac命令只在JDK中提供。

Q3: Linux下配置环境变量后仍不生效

A: 确认是否执行了source命令, 或检查配置文件语法是否正确。

实际案例

案例: 某企业开发环境标准化配置

某互联网公司要求所有开发人员使用统一的JDK版本 (JDK 1.8.0_281)。开发团队制作了自动化脚本, 新入职员工运行脚本后自动完成JDK安装和环境配置。脚本会检测系统类型 (Windows/Linux/Mac), 自动下载指定版本JDK, 配置JAVA_HOME和Path, 并验证安装结果。通过标准化配置, 避免了因环境差异导致的"在我机器上能运行"的问题。

1.2 Java程序基本结构

知识点讲解

Java程序的基本结构包括包声明、导入语句、类定义和主方法。理解程序结构是编写正确Java代码的基础。每个Java源文件以.java为扩展名, 文件名必须与公共类名完全一致 (包括大小写)。一个源文件中可以有多个类, 但最多只能有一个public类。

操作步骤

编写第一个Java程序的步骤:

1. 创建文件 HelloWorld.java
2. 编写代码:

```
public class HelloWorld { public static void main(String[] args) { System.out.println("Hello, World!"); }}
```
3. 编译: `javac HelloWorld.java` (生成HelloWorld.class字节码文件)
4. 运行: `java HelloWorld` (注意不带.class后缀)

注意事项

注意:

- main方法是Java程序的入口, 必须是 `public static void main(String[] args)`
- 类名首字母大写, 采用驼峰命名法 (如HelloWorld)
- 每条语句以分号;结尾
- Java是大小写敏感的语言, HelloWorld和helloworld是不同的标识符
- 编译时文件名要带.java后缀, 运行时只写类名

常见问题

Q1: 编译报错"类HelloWorld是公共的, 应在名为HelloWorld.java的文件中声明"

A: public类的类名必须与文件名完全一致，检查大小写是否匹配。

Q2: 运行时报错"找不到或无法加载主类"

A: 检查类路径是否正确，是否在正确的目录下执行，以及类名是否拼写正确。

Q3: main方法写错导致程序无法运行

A: main方法必须是public static void main(String[] args)，参数类型必须是String数组。

实际案例

案例：企业级项目包结构规范

某电商公司规定所有项目必须遵循以下包结构规范：com.company.project.module，例如：com.alibaba.order.service。这种结构保证了代码的组织性和可维护性，避免了类名冲突。开发规范要求：所有业务代码放在module包下，按功能分层为controller、service、dao、entity等子包。

第2章 Java基础语法

2.1 数据类型与变量

知识点讲解

Java是强类型语言，每个变量都必须声明其数据类型。数据类型分为基本数据类型和引用数据类型两大类。**基本数据类型（8种）**：整数型：byte(1字节)、short(2字节)、int(4字节)、long(8字节)；浮点型：float(4字节)、double(8字节)；字符型：char(2字节，Unicode编码)；布尔型：boolean（true/false）。**引用数据类型**：类、接口、数组、枚举、注解。

操作步骤

变量声明与使用流程：

- 声明变量：数据类型 变量名；（如 int age;）
- 初始化变量：变量名 = 初始值；（如 age = 25;）
- 声明并初始化：数据类型 变量名 = 初始值；（如 int age = 25;）
- 使用变量进行运算或输出

```
int age = 25;
double salary = 8500.50;
char grade = 'A';
boolean isActive = true;
String name = "张三";
System.out.println("姓名：" + name + "，年龄：" + age);
```

注意事项

注意：

- long类型赋值时，数值后需加L或l（建议大写L，避免与数字1混淆）
- float类型赋值时，数值后需加F或f，否则默认为double类型
- char类型使用单引号，String类型使用双引号
- 变量必须先声明后使用，且在同一作用域内不能重复声明
- 变量命名规范：首字母小写，驼峰命名（如studentName）

常见问题

Q1：int类型存储大数时溢出

A：int最大值为2,147,483,647，超过此值需使用long类型。计算前注意数值范围。

Q2：浮点数运算精度丢失

A：float和double在二进制表示时存在精度问题。涉及金额计算时，应使用BigDecimal类。

Q3：类型转换导致数据丢失

A：大类型转小类型（如int转byte）需要强制转换，可能丢失数据。小类型转大类型自动完成。

实际案例

案例：电商系统商品信息管理

某电商平台商品信息的数据类型设计：商品ID使用long（支持海量商品），商品名称使用String，价格使用BigDecimal（精确计算，避免浮点误差），库存数量使用int，是否上架使用boolean，上架时间使用long（时间戳）。开发规范要求：所有金额字段必须使用BigDecimal，禁止使用float或double，以保证财务计算的精确性。

2.2 运算符与表达式

知识点讲解

运算符是进行数据操作的符号。Java运算符分为：算术运算符、赋值运算符、比较运算符、逻辑运算符、位运算符、三元运算符。**运算符优先级（从高到低）**：括号 > 一元运算符 > 算术运算符 > 移位运算符 > 比较运算符 > 逻辑运算符 > 三元运算符 > 赋值运算符。

操作步骤

常用运算符使用规范：

1. 算术运算：+、-、*、/、%（取模）
2. 自增自减：++、--（前置先增减后使用，后置先使用后增减）
3. 比较运算：==、!=、>、<、>=、<=（结果为boolean）
4. 逻辑运算：&&（短路与）、||（短路或）、！（非）
5. 三元运算：条件 ? 表达式1 : 表达式2

```
int a = 10, b = 3;
int sum = a + b;      // 13
int mod = a % b;      // 1
boolean result = (a > 5) && (b < 5); // true
int max = (a > b) ? a : b; // 10
```

注意事项

注意：

- 整数相除结果仍为整数，如10/3=3，需要小数应转换为浮点类型
- ++i和i++在单独使用时效果相同，在表达式中结果不同
- ==比较基本类型时比较值，比较引用类型时比较内存地址
- 字符串比较使用equals()方法，不能使用==
- 逻辑运算符&&和||具有短路特性：左边能确定结果时右边不执行

常见问题

Q1: 10/3结果为3而不是3.33

A: 整数除法结果截断小数。应写成10.0/3或(double)10/3。

Q2: 比较两个字符串内容相等使用==返回false

A: ==比较的是引用地址。字符串内容比较应使用str1.equals(str2)。

Q3: 逻辑运算中右侧表达式未执行导致异常

A: 利用短路特性时需注意，如obj != null && obj.method()是安全的写法。

实际案例

案例：权限系统位运算设计

某企业管理系统使用位运算进行权限控制：定义PERMISSION_READ=1、PERMISSION_WRITE=2、PERMISSION_DELETE=4、PERMISSION_ADMIN=8。组合权限使用按位或：userPermission = PERMISSION_READ | PERMISSION_WRITE。检查权限使用按位与：canDelete = (userPermission & PERMISSION_DELETE) != 0。这种设计节省存储空间，一个int可存储32种权限状态，且运算效率极高。

第3章 流程控制

3.1 条件语句

知识点讲解

条件语句用于根据不同条件执行不同代码块。Java提供if语句、if-else语句、if-else if-else语句和switch语句。if语句适用于范围判断和复杂条件，switch语句适用于等值判断（JDK 7+支持String类型）。

操作步骤

if-else条件判断流程：

1. 确定判断条件
2. 编写if(条件) { 执行代码 }
3. 需要备选方案时添加else { 备选代码 }
4. 多条件判断使用else if

```
int score = 85;
if (score >= 90) {
    System.out.println("优秀");
} else if (score >= 80) {
    System.out.println("良好");
} else if (score >= 60) {
    System.out.println("及格");
} else {
    System.out.println("不及格");
}
```

switch语句使用流程：

1. 确定判断变量（支持byte、short、int、char、String、枚举）
2. 编写switch(变量) { case 值: 代码; break; ... }
3. 添加default处理未匹配情况

```
String day = "Monday";
switch (day) {
    case "Monday":
        System.out.println("星期一");
        break;
    case "Tuesday":
        System.out.println("星期二");
        break;
    default:
        System.out.println("其他");
}
```

注意事项

注意：

- if条件必须是boolean类型，不能是整数（区别于C语言）
- switch的case值必须是常量，且不能重复
- switch中忘记写break会导致"穿透"，执行后续case
- 多条件判断时，将概率高的条件放在前面，提高效率
- JDK 14+支持switch表达式，可简化代码

常见问题

Q1: if(score = 90)编译报错

A: 条件判断使用==而非=。=是赋值，==是相等比较。

Q2: switch中case穿透导致意外执行

A: 每个case末尾必须加break，除非有意利用穿透特性（如多个case共享代码）。

Q3: switch不支持long类型

A: switch仅支持byte、short、int、char及对应的包装类、String、枚举。long需用if判断。

实际案例

案例：电商订单状态流转控制

某电商系统订单状态管理使用switch处理状态流转：CREATED状态验证订单并更新为PAID；PAID状态准备发货并更新为SHIPPED；SHIPPED状态确认收货并更新为DELIVERED；DELIVERED状态自动确认。default抛出IllegalArgumentException处理未知状态。

3.2 循环语句

知识点讲解

循环语句用于重复执行代码块。Java提供for循环、while循环、do-while循环，以及增强for循环（foreach）。for循环适用于已知循环次数的场景，while循环适用于条件控制的场景，do-while至少执行一次。

操作步骤

三种循环的使用场景：

1. **for循环**：已知次数，如遍历数组、执行固定次数操作
2. **while循环**：条件控制，如读取文件直到EOF、等待特定条件
3. **do-while循环**：至少执行一次，如菜单选择、用户输入验证

```
// for循环示例：计算1到100的和
int sum = 0;
for (int i = 1; i <= 100; i++) {
    sum += i;
}
```

```
// while循环示例：读取数据直到满足条件
while (dataQueue.hasNext()) {
    process(dataQueue.next());
}

// 增强for循环：遍历集合
for (String item : itemList) {
    System.out.println(item);
}
```

注意事项

注意：

- 避免死循环：确保循环条件最终会变为false
- break用于跳出当前循环，continue用于跳过当前迭代
- 嵌套循环时注意性能，双重循环时间复杂度为 $O(n^2)$
- 增强for循环不能修改集合结构（增删元素），会报ConcurrentModificationException
- 循环变量尽量在for语句中声明，减少作用域污染

常见问题

Q1：循环条件写错导致死循环

A：检查循环变量是否在循环体内被正确更新。如for(int i=0; i<10; i++)中i++不能遗漏。

Q2：在增强for循环中删除元素报错

A：使用Iterator的remove()方法，或转换为普通for循环倒序删除。

Q3：嵌套循环中break只跳出内层循环

A：如需跳出外层循环，可使用标签（label）或在外层循环条件中设置标志位。

实际案例

案例：批量数据处理优化

某金融系统需要处理百万级交易记录，采用分页循环处理：设置pageSize=1000，计算总页数，每页查询处理后清空列表。这种分页循环处理方式避免了全量加载数据导致的内存溢出（OOM）问题。

第4章 数组与字符串

4.1 数组的定义与使用

知识点讲解

数组是相同数据类型元素的集合，在内存中连续存储。Java数组是引用类型，长度固定，声明后不可改变。数组可以是一维、二维或多维。Java支持不规则数组（每行长度不同）。

操作步骤

数组声明与使用流程：

1. 声明数组：数据类型[] 数组名;（推荐）或 数据类型 数组名[];
2. 分配空间：数组名 = new 数据类型[长度];
3. 初始化：直接赋值或使用循环
4. 访问元素：数组名[索引]（索引从0开始）

```
// 数组声明与初始化
int[] scores = new int[5];
int[] numbers = {10, 20, 30, 40};
int[][] matrix = new int[3][4];

// 遍历数组
for (int i = 0; i < scores.length; i++) {
    System.out.println(scores[i]);
}

// 使用Arrays工具类
Arrays.sort(numbers);
int index = Arrays.binarySearch(numbers, 30);
```

注意事项

注意：

- 数组索引从0开始，最大索引为length-1
- 访问越界会抛出ArrayIndexOutOfBoundsException
- 数组长度固定，需要动态扩容时应使用ArrayList
- 数组是引用类型，赋值操作是引用传递，不是复制
- 二维数组的每一行长度可以不同（不规则数组）

常见问题

Q1: ArrayIndexOutOfBoundsException异常

A: 检查循环条件，确保 $i < \text{array.length}$ ，而不是 $i \leq \text{array.length}$ 。

Q2：数组赋值后修改影响原数组

A：数组是引用类型，赋值后两个变量指向同一内存。复制数组使用Arrays.copyOf()或clone()。

Q3：二维数组matrix[0].length获取列数

A：二维数组每行长度可能不同，获取某行列数应使用matrix[row].length。

实际案例

案例：学生成绩统计分析系统

某学校成绩管理系统使用二维数组存储班级学生成绩：3个班级，每个班级不同人数。通过嵌套循环计算每个班级的总分和平均分。使用不规则数组灵活处理不同规模的班级数据。

4.2 String类的常用操作

知识点讲解

String是Java中使用最频繁的类之一，用于表示不可变字符序列。String类被final修饰，不可继承，内容不可修改。字符串常量存储在字符串常量池中，相同内容的字符串共享同一对象。频繁修改字符串时应使用StringBuilder或StringBuffer。

操作步骤

String常用操作方法：

1. 创建字符串：直接赋值、new String()、String.valueOf()
2. 获取长度：length()
3. 截取子串：substring(beginIndex, endIndex)
4. 查找位置：indexOf()、lastIndexOf()
5. 替换内容：replace()、replaceAll()（支持正则）
6. 分割字符串：split()（支持正则）
7. 去除空白：trim()
8. 大小写转换：toUpperCase()、toLowerCase()
9. 比较内容：equals()、equalsIgnoreCase()、compareTo()
10. 判断是否包含：contains()、startsWith()、endsWith()

```
String str = " Hello, Java World! ";
str = str.trim();
int len = str.length();
String sub = str.substring(7, 11);
boolean contains = str.contains("World");
String replaced = str.replace("Java", "Python");
String[] parts = str.split(",");
```

注意事项

注意：

- String是不可变的，每次修改都会创建新对象
- 字符串比较使用equals()，==比较的是引用地址
- 频繁拼接字符串时，优先使用StringBuilder（线程不安全，效率高）
- 多线程环境下频繁修改字符串使用StringBuffer（线程安全）
- split()参数是正则表达式，特殊字符需要转义（如\.

常见问题

Q1：字符串拼接效率低

A：循环中避免使用+拼接字符串，应使用StringBuilder的append()方法。

Q2：split(".")结果为空数组

A：.在正则中表示任意字符，应使用split("\\.")进行转义。

Q3：字符串比较==和equals结果不同

A：==比较内存地址，equals比较内容。new String("a")和"a"用==比较结果为false。

实际案例

案例：日志解析与敏感信息过滤

某系统需要处理用户日志，过滤敏感信息（手机号、身份证号）：使用replaceAll("(1\\d{2})\\d{4}(\\d{4})", "\$1****\$2")对手机号脱敏，使用replaceAll("(\\d{6})\\d{8}(\\d{4})", "\$1*****\$2")对身份证号脱敏。使用正则表达式和replaceAll实现高效的信息脱敏处理。

第5章 面向对象编程基础

5.1 类与对象

知识点讲解

面向对象编程（OOP）是Java的核心思想，通过类和对象来组织代码。类是对象的模板，对象是类的实例。类包含属性（成员变量）和方法（成员函数），通过封装将数据和行为结合在一起。创建对象使用new关键字调用构造方法。

操作步骤

类的定义与对象创建流程：

1. 定义类：使用class关键字，包含属性和方法
2. 定义构造方法：与类同名，无返回值，可重载
3. 创建对象：类名 对象名 = new 构造方法(参数);
4. 访问成员：对象名.属性 或 对象名.方法()

```
// 类定义示例
public class Employee {
    private String name;
    private int age;
    private double salary;

    public Employee(String name, int age, double salary) {
        this.name = name;
        this.age = age;
        this.salary = salary;
    }

    public void raiseSalary(double percent) {
        this.salary *= (1 + percent);
    }

    public String getInfo() {
        return name + ", 年龄: " + age + ", 薪资: " + salary;
    }
}

// 创建对象
Employee emp = new Employee("张三", 28, 8000);
emp.raiseSalary(0.1);
System.out.println(emp.getInfo());
```

注意事项

注意：

- 类名首字母大写，采用驼峰命名法

- 构造方法没有返回值类型，连void也不能写
- 未定义构造方法时，编译器提供默认无参构造方法
- 一旦定义了有参构造方法，默认无参构造方法消失，如需使用需显式定义
- this关键字指代当前对象，区分成员变量和局部变量

常见问题

Q1: new对象后属性值为null或0

A: 未在构造方法中初始化属性，对象属性会有默认值（引用类型为null，数值为0）。

Q2: 构造方法写了返回值类型导致编译错误

A: 构造方法不能有返回值类型，方法名必须与类名完全一致。

Q3: 默认构造方法消失导致子类编译错误

A: 父类定义了有参构造方法后，子类构造方法默认调用super()会报错，需在父类显式定义无参构造方法。

实际案例

案例：企业人力资源管理系统实体设计

某企业HR系统员工类设计规范：员工编号使用Long，工号使用String，姓名使用String，所属部门使用Department对象关联，入职日期使用LocalDate，职级使用枚举类型JobLevel。通过面向对象设计，将员工属性与业务行为封装在一起，提高代码内聚性。

5.2 封装、继承与多态

知识点讲解

封装、继承、多态是面向对象的三大特性。封装是将数据隐藏，通过公共方法访问，提高安全性和可维护性。继承是子类继承父类的属性和方法，实现代码复用，使用extends关键字。多态是同一操作作用于不同对象产生不同行为，包括编译时多态（方法重载）和运行时多态（方法重写）。

操作步骤

三大特性的实现步骤：

1. **封装**：属性私有化（private），提供getter/setter方法
2. **继承**：子类extends父类，可重写父类方法，可添加新属性和方法
3. **多态**：父类引用指向子类对象，调用重写方法时执行子类版本

```
// 封装示例
public class BankAccount {
    private double balance;
    public double getBalance() { return balance; }
    public void deposit(double amount) {
        if (amount > 0) { this.balance += amount; }
    }
}
```

```
// 继承与多态示例
public class Animal {
    public void speak() { System.out.println("动物叫声"); }
}
public class Dog extends Animal {
    @Override
    public void speak() { System.out.println("汪汪汪"); }
}
public class Cat extends Animal {
    @Override
    public void speak() { System.out.println("喵喵喵"); }
}

// 多态使用
Animal animal1 = new Dog();
Animal animal2 = new Cat();
animal1.speak(); // 汪汪汪
animal2.speak(); // 喵喵喵
```

注意事项

注意：

- 封装不是必须所有属性都private，根据访问控制需求选择合适修饰符
- Java只支持单继承，一个类只能直接继承一个父类
- 方法重写（Override）要求方法名、参数、返回值相同，访问权限不能更严格
- 多态中，父类引用不能调用子类特有方法，需要向下转型（instanceof判断安全）
- 构造方法不能被继承，子类通过super()调用父类构造方法

常见问题

Q1：子类无法访问父类private成员

A：private仅在本类可见。子类应通过父类提供的protected或public方法访问。

Q2：方法重写时参数类型不同导致不是重写

A：方法重写要求参数列表完全一致（类型、个数、顺序）。参数不同属于方法重载（Overload）。

Q3：向下转型时抛出ClassCastException

A：转型前使用instanceof判断对象类型，确保类型兼容后再转型。

实际案例

案例：电商平台支付系统设计

某电商平台支持多种支付方式（支付宝、微信、银行卡），使用多态实现统一支付接口：定义抽象类Payment，包含pay()和refund()抽象方法。各支付方式继承Payment实现具体逻辑。统一调用processPayment(Payment payment, double amount)处理支付，新增支付方式时只需添加子类，无需修改原有代码，符合开闭原则。

第6章 面向对象高级特性

6.1 抽象类与接口

知识点讲解

抽象类和接口是Java实现抽象化的两种机制，用于定义规范，由子类或实现类完成具体功能。抽象类使用abstract修饰，可包含抽象方法和具体方法，可有成员变量，单继承。接口使用interface定义，JDK 8前只能有抽象方法，JDK 8+可有默认方法和静态方法，多实现。

操作步骤

抽象类与接口的定义和使用：

1. **定义抽象类：**abstract class 类名 { abstract 返回类型 方法名(); }
2. **定义接口：**interface 接口名 { 返回类型 方法名(); }
3. **继承抽象类：**class 子类 extends 抽象类 { 实现所有抽象方法 }
4. **实现接口：**class 类 implements 接口1, 接口2 { 实现所有方法 }

```
// 抽象类示例
public abstract class Shape {
    protected String color;
    public abstract double getArea();
    public void setColor(String color) { this.color = color; }
}

// 接口示例
public interface Flyable {
    void fly();
    default void land() { System.out.println("降落"); }
}

// 实现
public class Bird extends Animal implements Flyable {
    @Override
    public void fly() { System.out.println("鸟儿飞翔"); }
}
```

注意事项

注意：

- 抽象类不能直接实例化，但可以有构造方法供子类调用
- 接口中的方法默认是public abstract，变量默认是public static final
- JDK 8接口支持default方法，JDK 9支持private方法
- 接口之间可以继承，使用extends，且支持多继承
- 选择抽象类还是接口：is-a关系用抽象类，can-do关系用接口

常见问题

Q1: 抽象类没有抽象方法是否可以

A: 可以，但失去抽象类的意义。抽象类主要用于被继承，不能直接new。

Q2: 实现接口时方法访问权限问题

A: 接口方法默认public，实现时不能降低访问权限，必须声明为public。

Q3: 接口和抽象类的选择困惑

A: 需要多继承时用接口；需要代码复用（有具体方法）时用抽象类。实际开发中常结合使用。

实际案例

案例：微服务架构中的服务接口设计

某微服务系统定义统一的服务接口规范：基础服务接口BaseService定义增删改查方法，用户服务接口UserService继承BaseService并扩展登录、重置密码等方法。通过接口定义服务契约，实现类完成具体逻辑，便于单元测试和Mock测试。

6.2 异常处理

知识点讲解

异常是程序运行过程中出现的非正常情况。Java异常处理机制使用try-catch-finally结构，将正常逻辑与错误处理分离。异常体系：Throwable → Error（系统级错误，不处理）/ Exception → RuntimeException（运行时异常，不强制处理）/ 其他Exception（受检异常，必须处理）。

操作步骤

异常处理的标准流程：

1. 将可能抛出异常的代码放在try块中
2. 使用catch块捕获并处理特定异常
3. 需要始终执行的清理代码放在finally块中
4. 方法声明中使用throws声明可能抛出的受检异常
5. 使用throw主动抛出异常

```
// 异常处理示例
try {
    int result = 10 / 0;
    FileReader reader = new FileReader("file.txt");
} catch (ArithmeticException e) {
    System.out.println("除数不能为0 : " + e.getMessage());
} catch (IOException e) {
    System.out.println("文件读取失败 : " + e.getMessage());
} catch (Exception e) {
    System.out.println("未知异常 : " + e.getMessage());
} finally {
    System.out.println("无论是否异常，都会执行");
}
```

```
// 自定义异常
public class BusinessException extends RuntimeException {
    public BusinessException(String message) { super(message); }
}

// 抛出异常
if (amount < 0) {
    throw new BusinessException("金额不能为负数");
}
```

注意事项

注意：

- catch块应按子类到父类顺序排列，否则编译错误
- finally块中的代码始终执行（除非System.exit()或JVM崩溃）
- 不要在finally块中写return，会覆盖try/catch中的return值
- 受检异常（Checked Exception）必须处理或声明throws
- 运行时异常（RuntimeException）通常由程序逻辑错误引起，应通过代码避免

常见问题

Q1: catch中e.printStackTrace()直接输出到控制台

A: 生产环境应使用日志框架（如SLF4J/Logback）记录异常，便于问题追踪。

Q2: 吞掉异常（catch块为空）

A: 绝对禁止空catch块，至少记录日志。吞掉异常会导致问题难以排查。

Q3: 在循环中频繁创建异常对象影响性能

A: 异常创建代价高，应避免将异常用于正常流程控制。可预见的条件用if判断。

实际案例

案例：银行转账事务异常处理

某银行核心系统转账操作的异常处理设计：try块中开启事务、检查余额、执行转账、记录流水、提交事务。catch InsufficientBalanceException时回滚事务并记录警告日志。catch Exception时回滚事务并记录错误日志，向上抛出SystemException给用户友好提示。finally块关闭事务连接。通过分层异常处理，区分业务异常和系统异常。

第7章 集合框架

7.1 List与Set集合

知识点讲解

Java集合框架（Java Collections Framework）提供了一套标准的接口和实现类，用于存储和操作对象组。List是有序、可重复的集合。主要实现：ArrayList（基于数组，查询快）、LinkedList（基于链表，增删快）、Vector（线程安全，已过时）。Set是无序、不可重复的集合。主要实现：HashSet（基于哈希表，无序）、LinkedHashSet（保持插入顺序）、TreeSet（基于红黑树，自动排序）。

操作步骤

List与Set的常用操作：

1. 创建集合：List<String> list = new ArrayList<>();
2. 添加元素：add()、addAll()
3. 获取元素：get(index)（List特有）
4. 删除元素：remove()、removeIf()
5. 遍历集合：for-each、Iterator、forEach()
6. 集合转换：toArray()、stream().map()

```
// List操作示例
List<String> list = new ArrayList<>();
list.add("Java");
list.add("Python");
list.add(1, "C++");
String first = list.get(0);
list.remove("Python");

// Set操作示例（自动去重）
Set<Integer> set = new HashSet<>();
set.add(10); set.add(20); set.add(10);
System.out.println(set.size()); // 2

// TreeSet自动排序
Set<Integer> sortedSet = new TreeSet<>();
sortedSet.addAll(Arrays.asList(5, 2, 8, 1));
System.out.println(sortedSet); // [1, 2, 5, 8]
```

注意事项

注意：

- ArrayList初始容量为10，扩容时增长1.5倍，大量数据时建议指定初始容量
- HashSet判断重复依据是hashCode()和equals()，自定义对象需重写这两个方法
- TreeSet要求元素实现Comparable接口，或在构造时传入Comparator
- 多线程环境下使用Collections.synchronizedList()或CopyOnWriteArrayList

- 避免在遍历过程中直接修改集合结构（增删元素），应使用Iterator的remove()

常见问题

Q1: HashSet中自定义对象没有去重

A: 自定义类必须重写hashCode()和equals()方法，且两者逻辑一致。

Q2: ArrayList频繁增删性能差

A: ArrayList中间插入/删除需要移动后续元素。频繁增删场景应使用LinkedList。

Q3: ConcurrentModificationException异常

A: 遍历时修改集合结构导致。使用Iterator.remove()或CopyOnWriteArrayList解决。

实际案例

案例：电商商品去重与排序

某电商系统商品推荐去重和排序实现：从浏览历史、相似用户、热门商品三个渠道获取推荐列表。使用LinkedHashSet去重并保持推荐优先级顺序。转换为ArrayList后按热度分数降序排序，截取前20条返回。LinkedHashSet既保证去重又保持插入顺序，满足业务需求。

7.2 Map集合与泛型

知识点讲解

Map用于存储键值对（key-value），键不可重复，值可重复。主要实现：HashMap（基于哈希表）、LinkedHashMap（保持插入顺序）、TreeMap（按键排序）、ConcurrentHashMap（线程安全）。泛型（Generics）是JDK 5引入的特性，用于类型参数化，提供编译时类型检查，避免强制类型转换。

操作步骤

Map与泛型的使用流程：

1. 创建Map：Map<K, V> map = new HashMap<>();
2. 添加键值对：put(key, value)
3. 获取值：get(key)
4. 判断包含：containsKey()、containsValue()
5. 遍历：entrySet()、keySet()、forEach()
6. 删除：remove(key)

```
// Map操作示例
Map<String, Integer> scores = new HashMap<>();
scores.put("张三", 90);
scores.put("李四", 85);
scores.put("张三", 95);

Integer zhangScore = scores.get("张三");
boolean hasWang = scores.containsKey("王五");
```

```
for (Map.Entry<String, Integer> entry : scores.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}

// 泛型类示例
public class Box<T> {
    private T content;
    public void set(T content) { this.content = content; }
    public T get() { return content; }
}
```

注意事项

注意：

- HashMap键和值都可以为null，但键只能有一个null
- HashMap非线程安全，多线程使用ConcurrentHashMap
- 自定义对象作为HashMap键时，必须重写hashCode()和equals()
- 泛型只在编译时有效，运行时会类型擦除（Type Erasure）
- 不能创建泛型数组：new T[10]是非法的
- 通配符? extends T表示上界，? super T表示下界（PECS原则）

常见问题

Q1: HashMap遍历时修改导致数据不一致

A: 使用Iterator遍历并删除，或遍历完成后统一处理。ConcurrentHashMap支持并发遍历。

Q2: 自定义对象作为Map键后无法获取值

A: 检查是否重写了hashCode()和equals()，且对象放入Map后是否修改了影响hashCode的字段。

Q3: 泛型类型擦除导致运行时类型信息丢失

A: 通过反射获取ParameterizedType，或使用TypeToken（如Gson的TypeToken）保留泛型信息。

实际案例

案例：系统配置缓存设计

某企业系统使用ConcurrentHashMap实现配置缓存：getConfig方法先从缓存获取，未命中则从数据库加载并放入缓存。refreshConfig方法清除指定缓存键，下次重新加载。使用ConcurrentHashMap保证线程安全，避免多线程环境下HashMap的死循环问题。

第8章 IO流与文件操作

8.1 字节流与字符流

知识点讲解

Java IO流用于数据的输入和输出操作。按数据单位分为字节流（8位）和字符流（16位，Unicode）。字节流包括InputStream/OutputStream，适用于二进制数据（图片、视频、可执行文件）。字符流包括Reader/Writer，适用于文本数据，自动处理字符编码转换。节点流直接操作数据源（FileInputStream、FileReader）。处理流包装节点流，提供额外功能（BufferedInputStream、BufferedReader）。

操作步骤

IO流的标准使用流程：

1. 确定数据源/目的地（文件、网络、内存等）
2. 选择合适的流类型（字节/字符、输入/输出）
3. 创建流对象，建议使用Buffered流提高性能
4. 读写数据
5. 关闭流（finally块或try-with-resources）

```
// 字节流复制文件（try-with-resources自动关闭）
try (InputStream in = new BufferedInputStream(new FileInputStream("src.jpg"));
     OutputStream out = new BufferedOutputStream(new FileOutputStream("dest.jpg"))) {
    byte[] buffer = new byte[1024];
    int len;
    while ((len = in.read(buffer)) != -1) {
        out.write(buffer, 0, len);
    }
} catch (IOException e) {
    logger.error("文件复制失败", e);
}

// 字符流读取文本文件（指定编码）
try (BufferedReader reader = new BufferedReader(
    new InputStreamReader(new FileInputStream("text.txt"), "UTF-8"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    logger.error("文件读取失败", e);
}
```

注意事项

注意：

- 始终使用Buffered流包装节点流，减少系统调用次数，提高性能

- 字符流必须指定编码，否则使用平台默认编码，可能导致乱码
- 使用try-with-resources（JDK 7+）自动关闭流，避免资源泄漏
- 字节流适用于所有类型文件，字符流仅适用于文本文件
- 大文件处理避免一次性读入内存，使用缓冲区逐块读取

常见问题

Q1：读取中文文件出现乱码

A：使用InputStreamReader指定正确的编码（如UTF-8），不要用FileReader（使用系统默认编码）。

Q2：未关闭流导致文件被占用无法删除

A：确保在finally块中关闭流，或使用try-with-resources语法。

Q3：大文件读取导致内存溢出

A：不要一次性读取整个文件到内存，使用缓冲区逐行或逐块读取。

实际案例

案例：日志文件分割与归档系统

某系统日志文件按大小自动分割，超过100MB时创建新文件：writeLog方法检查当前日志文件大小，超过阈值则调用archiveLog归档旧文件。使用BufferedWriter追加写入新日志，指定UTF-8编码。archiveLog方法使用时间戳重命名旧文件并移动到archive目录。

8.2 文件读写与序列化

知识点讲解

Java NIO（New IO）提供了更高效的IO操作方式，包括通道（Channel）、缓冲区（Buffer）和选择器（Selector）。对象序列化是将对象转换为字节序列，便于存储或网络传输。实现Serializable接口的类可被序列化。

操作步骤

NIO文件操作与对象序列化流程：

1. **NIO读取文件：**使用Paths.get()获取Path，Files.readAllLines()或newBufferedReader()
2. **NIO写入文件：**Files.write()或newBufferedWriter()
3. **对象序列化：**ObjectOutputStream.writeObject()
4. **对象反序列化：**ObjectInputStream.readObject()

```
// NIO读取文件 (JDK 7+)
List<String> lines = Files.readAllLines(
    Paths.get("data.txt"), StandardCharsets.UTF_8);

// NIO写入文件
Files.write(Paths.get("output.txt"), "Hello NIO".getBytes());
```



```
// 对象序列化
public class User implements Serializable {
    private static final long serialVersionUID = 1L;
    private String name;
    private transient String password;
}

try (ObjectOutputStream oos = new ObjectOutputStream(
    new FileOutputStream("user.dat"))) {
    User user = new User("张三", "123456");
    oos.writeObject(user);
}

// 对象反序列化
try (ObjectInputStream ois = new ObjectInputStream(
    new FileInputStream("user.dat"))) {
    User user = (User) ois.readObject();
}
```

注意事项

注意：

- 序列化类必须实现Serializable接口，并声明serialVersionUID
- transient关键字标记的字段不会被序列化
- 反序列化时，serialVersionUID必须匹配，否则抛出InvalidClassException
- 静态变量不属于对象，不会被序列化
- NIO的Files工具类简化了文件操作，但大文件仍应使用流式处理

常见问题

Q1：反序列化时报InvalidClassException

A：类结构变更后serialVersionUID不匹配。应显式声明serialVersionUID，避免依赖自动生成的值。

Q2：敏感信息被序列化到文件

A：使用transient关键字标记敏感字段，或自定义writeObject/readObject方法进行加密处理。

Q3：NIO的Files.readAllLines读取大文件内存溢出

A：大文件使用Files.lines()返回Stream，配合try-with-resources逐行处理。

实际案例

案例：分布式系统会话持久化

某分布式系统使用Redis存储会话，会话对象需要序列化：Session类实现Serializable，sessionId、userId、attributes等字段序列化，createTime标记为transient（由Redis管理TTL）。saveSession方法将对象序列化为字节数组存储到Redis，设置30分钟过期时间。

第9章 多线程编程

9.1 线程的创建与生命周期

知识点讲解

线程是程序执行的最小单位，多线程编程可充分利用多核CPU，提高程序并发处理能力。Java线程由JVM管理，底层映射到操作系统线程。线程生命周期：新建（New）→ 就绪（Runnable）→ 运行（Running）→ 阻塞（Blocked/Waiting/Timed Waiting）→ 终止（Terminated）。

操作步骤

创建线程的三种方式：

1. **继承Thread类**：class MyThread extends Thread { public void run() { ... } }
2. **实现Runnable接口**：class MyTask implements Runnable { public void run() { ... } }
3. **实现Callable接口（JDK 5+）**：可返回结果和抛出异常，配合FutureTask使用

```
// 方式1：继承Thread
class DownloadThread extends Thread {
    @Override
    public void run() {
        System.out.println("下载文件：" + Thread.currentThread().getName());
    }
}
DownloadThread t1 = new DownloadThread();
t1.start();

// 方式2：实现Runnable（推荐，避免单继承限制）
Runnable task = () -> {
    System.out.println("处理任务：" + Thread.currentThread().getName());
};
new Thread(task).start();

// 方式3：Callable + FutureTask
Callable<Integer> callable = () -> {
    TimeUnit.SECONDS.sleep(2);
    return 100;
};
FutureTask<Integer> futureTask = new FutureTask<>(callable);
new Thread(futureTask).start();
Integer result = futureTask.get();
```

注意事项

注意：

- 启动线程调用start()，不是run()。直接调用run()只是普通方法调用
- 实现Runnable更灵活，推荐优先使用。继承Thread无法继承其他类
- 线程优先级（setPriority）只是建议，实际调度由操作系统决定

- 线程名称应设置有意义的名称，便于调试和日志追踪
- 守护线程（setDaemon(true)）在JVM中所有非守护线程结束后自动终止

常见问题

Q1: 直接调用run()方法没有并发效果

A: 必须调用start()方法启动新线程。run()只是普通方法，在当前线程同步执行。

Q2: 线程启动后执行顺序不确定

A: 线程调度由操作系统决定，代码中不能依赖线程执行顺序。需要同步时使用锁或同步工具。

Q3: FutureTask.get()阻塞主线程

A: get()会阻塞直到任务完成。应使用带超时参数的get(timeout, unit)，或在任务完成后调用。

实际案例

案例：电商订单异步处理

某电商系统订单提交后需要执行多个异步操作（发送短信、更新库存、记录日志）：主线程保存订单后，启动三个线程分别执行短信通知、库存扣减、日志记录。实际生产环境应使用线程池（ThreadPoolExecutor）而非直接new Thread，避免资源耗尽。

9.2 线程同步与线程池

知识点讲解

多线程访问共享资源时，需要同步机制保证数据一致性。Java提供synchronized关键字、ReentrantLock、原子类等同步工具。线程池（ThreadPoolExecutor）用于管理和复用线程，减少线程创建销毁的开销，控制并发数量。

操作步骤

线程同步的常用方式：

1. **synchronized**：修饰方法或代码块，自动获取和释放锁
2. **ReentrantLock**：显式锁，支持公平锁、可中断、超时获取等高级功能
3. **原子类**：AtomicInteger、AtomicLong等，基于CAS无锁算法
4. **线程池创建**：Executors工厂方法或new ThreadPoolExecutor()

```
// synchronized示例
public class Counter {
    private int count = 0;
    public synchronized void increment() { count++; }
    public synchronized int getCount() { return count; }
}

// ReentrantLock示例
public class SafeCache {
    private final ReentrantLock lock = new ReentrantLock();
```

```
private Map<String, Object> cache = new HashMap<>();

public Object get(String key) {
    lock.lock();
    try {
        return cache.get(key);
    } finally {
        lock.unlock();
    }
}

// 线程池示例
ExecutorService executor = new ThreadPoolExecutor(
    5, 10, 60L, TimeUnit.SECONDS,
    new LinkedBlockingQueue<>(100),
    new ThreadFactoryBuilder().setNameFormat("order-pool-%d").build(),
    new ThreadPoolExecutor.CallerRunsPolicy()
);
```

注意事项

注意：

- synchronized锁的是对象（实例方法锁this，静态方法锁Class对象）
- ReentrantLock必须在finally块中释放，否则可能导致死锁
- 优先使用线程池，禁止在循环中直接new Thread()
- 线程池参数需根据业务场景调优：核心线程数、最大线程数、队列容量
- 避免在同步块中执行耗时操作（如IO、网络请求），会大幅降低并发性能

常见问题

Q1：synchronized方法导致性能瓶颈

A：缩小同步范围，只同步必要的代码块。或使用读写锁（ReadWriteLock）区分读操作和写操作。

Q2：线程池任务队列满导致拒绝

A：合理设置队列大小和拒绝策略。监控线程池状态，及时调整参数。

Q3：死锁问题排查困难

A：避免嵌套锁，统一加锁顺序。使用jstack工具分析线程dump，定位死锁位置。

实际案例

案例：秒杀系统库存扣减

某电商秒杀活动需要高并发扣减库存，使用乐观锁（CAS）避免性能瓶颈：SeckillService使用AtomicInteger存储库存，deductStock方法使用compareAndSet循环尝试扣减。使用固定大小线程池处理秒杀请求，AtomicInteger的CAS操作实现无锁并发，性能远高于synchronized。

